Kalya Naura Farabias
24/541690/TK/60097
ASD - C

Problem 1

Snippet A

```
// n >= 1
int sum = 0 ; // a
for (int i = 0 ; i < n ; i++) { // b
  for (int j = 0 ; j < i ; j++) { // c
    sum += arr [i] + arr [j] ; // d
  }
}
```

analisis line a
int sum = 0 → 1
analisis line b
int i = 0 → 1
i < n → n+1
i++ → n

analisis line c
int j = 0 → n
j < i → $\frac{n(n+1)}{2}$
j++ → $\frac{n(n-1)}{2}$

analisis line d
sum += arr [i] + arr [j]
→ array access (i & j) : 2
→ addition (arr i & arr j) : 1
→ addition (hasil penjumlahan array dijumlahkan ke nilai sum yang lama) : 1
→ assignment (=) : 1
→ jumlah eksekusi : $\frac{n(n-1)}{2}$

$$T(n) = 1+1+(n+1)+n+n+\left(\frac{n^2+n}{2}\right)+\left(\frac{n^2-n}{2}\right)+\left(\frac{5n^2-5n}{2}\right)$$

$$= 3n+3+\frac{7n^2-5n}{2}$$

$$= 3{,}5n^2+0{,}5n+3$$

Big - O class : $O(n^2)$

Best / worst case : best case = worst case. jumlah operasi hanya dipengaruhi oleh ukuran input n, bukan oleh variasi data di dalam input tersebut. Oleh karena itu, performa algoritma akan selalu sama untuk input dengan ukuran yang sama

Snippet B

```
// n ≥ 1 ; arr is a sorted array of n integers
int lo = 0 , hi = n - 1 , target = 42 ; // a
bool found = false ; // b
while (lo <= hi) { // c
  int mid = lo + (hi - lo) / 2 ; // d
  if (arr [mid] == target) { found = true ; break ; } // e
  else if (arr [mid] < target) { lo = mid + 1 ; } // f
  else { hi = mid - 1 ; } // g
}
```

analisis line a

lo = 0 → 1

n-1 → 1

hi = → 1

target = 42 → 1

deklarasi variabel baru → 1

Total cost : 5

analisis line b

bool found = false → 1

analisis line c

while (lo <= hi) → L+2

analisis line d

cost : 4

Jumlah eksekusi : L+1

Total : 4L+4

analisis line e

→ if (arr [mid] = = target)

cost : 2

Jumlah eksekusi : L+1

Total cost : 2L+2

→ found = true ; break ;

cost : 2

Jumlah eksekusi : 0

Total cost : 0

Analisis line f

→ else if (arr [mid] < target)

cost : 2

Jumlah eksekusi : L+1

Total cost : 2L+2

→ {lo = mid + 1 ; }

cost : 2

Jumlah eksekusi : L+1

Total cost : 2L+2

analisis line g

else { hi = mid - 1 ; }

cost : 2

Jumlah eksekusi : 0

Total cost : 0

$T(n) = 5 + 1 + (L+2) + (4L+4) + (2L+2) + (2L+2) + (2L+2)$

$= 11L + 18$

$= 11 \log_2 n + 18$ ▨

Big - O class : $O(\log n)$ ▨

Best / worst case

→ Best case {$O(1)$} : Terjadi jika target ditemukan tepat pada percobaan pertama (indeks tengah pertama). Algoritma langsung melakukan break dan keluar dari loop ▨

→ Worst case {$O(\log n)$} : Terjadi jika target berada di ujung data atau tidak ada sama sekali dalam array. Algoritma harus terus membagi rentang data menjadi dua bagian sampai rentangnya habis ▨

Snippet C

```
// n >= 1
int count = 0 ; // a
for (int i=lo ; i < n ; i++) { // b
    for (int j = i+1 ; j < n ; j++) { // c
      for ( int k = 0 ; k < n ; k += 2) { // d
        if (arr [i] != arr [j] && arr [k] > 0) // e
            count ++ ; // f
      }
    }
}
```

analisis line a

int count = 0 → 1

analisis line b

cost : 3

Jumlah eksekusi : $n+1$

Total cost : $3n+3$

analisis line c

cost : 3

jumlah eksekusi : $\frac{n(n+1)}{2}$

Total cost : $1{,}5n^2 + 1{,}5n$

analisis line d

cost : 3

jumlah eksekusi : $\frac{n(n-1)}{2} \times \left(\frac{n}{2}+1\right)$

Total cost : $0{,}25n^3 + 0{,}25n^2$

analisis line e

cost : 6

Jumlah eksekusi : $\frac{n(n-1)}{2} \times \frac{n}{2}$

Total cost : $1{,}25n^3 - 1{,}25n^2$

analisis line f

cost : 2

jumlah eksekusi : $\frac{n(n-1)}{2} \times \frac{n}{2}$

Total cost : $0{,}5n^3 - 0{,}5n^2$

$T(n) = 2{,}75n^3 + 0{,}25n^2 + 4{,}5n + 4$ ☑

Big-O class : $O(n^3)$ ☑

Best / worst case

- Best case : tetap dalam kelas $O(n^3)$ karena semua loop tetap berputar sampai habis, namun jumlah operasi sedikit berkurang karena baris 7 tidak pernah dijalankan ☑
- Worst case : $T(n)$ di atas (asumsi kondisi if selalu True, sehingga count += selalu dieksekusi) ☑

Problem 3 Part B

operation	Best case	worst case	Average case
Linear search (unsorted)	$O(1)$	$O(n)$	$O(n)$
Linear search (sorted, early exit)	$O(1)$	$O(n)$	$O(n)$
Binary search (sorted)	$O(1)$	$O(\log n)$	$O(\log n)$

☑

Explanation :

- Linear search (unsorted) : memeriksa setiap elemen satu per satu dari awal hingga akhir ☑
- Linear search (sorted) : tetap memeriksa satu per satu, meski bisa berhenti lebih awal jika target terlampaui ☑
- Binary search (sorted) : secara berulang membagi dua area pencarian, sehingga sangat efisien ☑

Analysis and Reasoning: binary search secara konsisten menjadi lebih efisien daripada linear search pada worst case mulai dari $n=4$. Pada $n=3$, linear search butuh maksimal 3 perbandingan. Binary search secara matematis butuh ≈ 2 perbandingan, namun karena ada overhead (seperti menghitung mid dan pengecekan kondisi loop), efisiensinya masih terasa setara. Pada $n=4$, linear search butuh maksimal 4 perbandingan. Sementara itu, Binary search hanya butuh maksimal 3 perbandingan ($\lfloor \log_2(4) \rfloor + 1$). Di titik inilah Binary search jelas unggul secara jumlah langkah kerja ☑

Problem 4 part A

Call #	value pushed	size after	capacity after	cost
1	10	1	2	1
2	20	2	2	1
3	30	3	4	3
4	40	4	4	1
5	50	5	8	5
6	60	6	8	1
7	70	7	8	1
8	80	8	8	1
9	90	9	16	9
10	100	10	16	1
11	110	11	16	1
12	120	12	16	1

Problem 4 part B

1. Total cost

$T_{12} = 1+1+3+1+5+1+1+1+9+1+1+1 = 26$

2. Average cost per call

$\frac{T_{12}}{12} = \frac{26}{12} \approx 2,17$

3. Formula Total Copy work & Geometric series

→ Append cost : selalu 1 untuk setiap elemen baru, totalnya adalah n

→ copy cost : terjadi saat n mencapai pangkat 2 (dimulai dari 2). Biaya lainnya adalah $2+4+8+\dots+2^k$, di mana 2^k adalah ukuran resize terakhir sebelum mencapai n

Evaluasi Deret Geometri

Total pekerjaan menyalin adalah deret geometri :

$S = 2+4+8+\dots+2^k$ → total copy work tidak akan pernah melebihi $2n$

Kesimpulan amortisasi :

karena total biaya T_n : (append) + (copy work) $\approx n+2n = 3n$, maka biaya rata-rata per panggilan adalah

$$\frac{T_n}{n} \approx \frac{3n}{n} = 3$$

Hal ini menunjukkan bahwa Amortized cost per call adalah $O(1)$ (konstan), karena total biaya tumbuh secara linear terhadap n, bukan kuadratik

Problem 4 part c

1. Initial capacity : 2

Strategy : if size == capacity, new capacity = old_capacity + 4

cost : 1 for append, 1 + old_size for resize

Call #	Value	Size After	Capacity After	Cost
1	10	1	2	1
2	20	2	2	1
3	30	3	6	3
4	40	4	6	1
5	50	5	6	1
6	60	6	6	1
7	70	7	10	7
8	80	8	10	1
9	90	9	10	1
10	100	10	10	1
11	110	11	14	11
12	120	12	14	1

2. Comparison of Total cost

- Doubling strategy (part A) : Total cost = 26
- Fixed increment (+4) strategy : Total cost = 30
- Conclusion : the fixed increment is more expensive (30 > 26). As n grows, this gap will widen significantly

3. Mathematical Derivation

a) doubling strategy (T(n)) :

- Resize happen at n = 2, 4, 8, 16, ... 2^k
- Total cost = $n + (2+4+8+\dots+n/2)$
- The sum of the geometric series is $\approx n$
- $T(n) = n + n = 2n$, which is $O(n)$
- Amortized cost : $2n/n = O(1)$

b) fixed increment strategy (T(n)) :

- Resizes happen every k steps
- Total copy work = $2+6+8+10+14+\dots+(n-k)$
- This is an arithmetic series. The sum of an arithmetic series is $O(n^2)$
- Total cost $T(n) = n +$ sum of arithmetic series $= O(n^2)$
- Amortized cost : $O(n^2)/n = O(n)$